A Multi-Scale Temporal Feature Aggregation Convolutional Neural Network for Portfolio Management

Si Shi, Jianjun Li*
School of Computer Science and Technology, Huazhong University of Science and Technology, China
{shisi_cc,jianjunli}@hust.edu.cn

Guohui Li
School of Software Engineering, Huazhong University of Science and Technology, China
guohuili@hust.edu.cn

Peng Pan
School of Computer Science and Technology, Huazhong University of Science and Technology, China
panpeng@hust.edu.cn

ABSTRACT

Financial portfolio management is the process of periodically reallocating a fund into different financial investment products, with the goal of achieving the maximum profits. While conventional financial machine learning methods try to predict the price trends, reinforcement learning based portfolio management methods makes trading decisions according to the price changes directly. However, existing reinforcement learning based methods are limited in extracting the price change information at single-scale level, which makes their performance still not satisfactory. In this paper, inspired by the Inception network that has achieved great success in computer vision and can extract multi-scale features simultaneously, we propose a novel Ensemble of Identical Independent Inception (EI^3) convolutional neural network, with the objective of addressing the limitation of existing reinforcement learning based portfolio management methods. With EI^3, multiple assets can be processed independently while sharing the same network parameters. Moreover, price movement information for each product can be extracted at multiple scales via wide network and then aggregated to make trading decision. Based on EI^3, we further propose a recurrent reinforcement learning framework to provide a deep machine learning solution for the portfolio management problem. Comprehensive experiments on the cryptocurrency datasets demonstrate the superiority of our method over existing competitors, in both upswing and downswing environments.

CCS CONCEPTS

• **Computing methodologies** → **Reinforcement learning**; • **Applied computing** → *Economics*.

*Jianjun Li is the corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CIKM '19, November 3–7, 2019, Beijing, China
© 2019 Association for Computing Machinery.
ACM ISBN 978-1-4503-6976-3/19/11…$15.00
https://doi.org/10.1145/3357384.3357961

KEYWORDS

Portfolio management, Reinforcement learning, Convolution neural network, Inception network

ACM Reference Format:
Si Shi, Jianjun Li, Guohui Li, and Peng Pan. 2019. A Multi-Scale Temporal Feature Aggregation Convolutional Neural Network for Portfolio Management. In *The 28th ACM International Conference on Information and Knowledge Management (CIKM '19), November 3–7, 2019, Beijing, China.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3357384.3357961

1 INTRODUCTION

Portfolio management is a decision-making process in which a certain amount of funds are continuously redeployed into a number of different financial investment products, aiming at maximizing returns while suppressing risks [28]. How to adjust portfolio weights periodically is the key problem in portfolio management. During the past few decades, the portfolio management problem has been widely studied and various methods have been proposed[25]. Recently, with the advance of artificial intelligence, machine learning has also been utilized in financial domain [8, 36], mainly for price or trend prediction with supervised learning. To address the portfolio management problem, portfolio still needs to be constructed according to the prediction. The performance of these predication-based methods mainly depends on the prediction accuracy, which is difficult to guarantee in noisy, highly volatile and non-stationary trading market.

To address the limitations of prediction-based methods, reinforcement learning (RL) in financial domain [30] combines "prediction" and "decision-making" in one step, and makes trading decisions straightforwardly according to the price change. For example, in the ensemble of identical independent elevator (EIIE) convolutional neural network [22], price movement of each asset is first extracted independently, and then normalized to generate the next adjusted portfolio weights. In this way, high precision prediction is not required, and manual transformation from predictions to actions can also be avoided. Moreover, by reinforcement learning, it is convenient to take external influence factors, such as transaction costs, investor's degree of risk-aversion [2], fundamentals, emergencies and public sentiment into consideration.

Though reinforcement learning in portfolio management has made significant progress, the RL-based methods are limited in extracting the price change information at single-scale level in a time window. But in real-life financial technical

analysis, traders usually need to consider temporal information at multiple scales. For example, indicators like MACD (moving average convergence/divergence) [3] uses a "fast" (short period) exponential moving average (EMA) and a "slow" (longer period) exponential moving average to produce trading signals. This reveals that information at multiple scales can help identify short-term fluctuation and long-term trend, and thus is useful in making trading decisions.

Recently, deeper and wider convolutional neural networks (CNNs) have made great contribution in computer vision. Among them, Inception networks [35] processes vision information with filters of multiple sizes operating on the same level, so that vision features from different scales can be used simultaneously. In this way, Inception network can achieve high precision in image classification. Inspired by its success, we propose to use the idea of Inception network to extract multi-scale temporal features, so as to address the limitation of existing RL-based portfolio management methods. Specifically, we propose a novel Ensemble of Identical Independent Inception (EI^3) convolutional neural network, which combines the advantages of both independent and identical network flow in EIIE [22] and multi-scale feature extraction in Inception network. The proposed EI^3 network first extracts multi-scale temporal features via several branches, then aggregates the features with the previous portfolio weights, and finally normalizes the assets to obtain the rebalanced portfolio weights. The main contributions of this work can be summarized as follows:

- We propose a multi-scale temporal feature aggregation convolution framework namely EI^3, which consists of two CNN branches extracting short-term and medium-term trends respectively, and a max pooling branch extracting highest price information. EI^3 is capable of capturing temporal patterns at different scales in price movement, while keeping different assets independent. Based on EI^3, we further propose a recurrent reinforcement learning framework to solve the portfolio management problem.
- We perform experiments on real-world datasets collected from Poloniex to evaluate the performance of our method versus both state-of-the-art RL-based methods and traditional portfolio management approaches. Experimental results show that in upswing and smooth fluctuation environments, our method achieves the highest profit than all the other methods. In downswing environment, our method learns to hold only cash to avoid risk and achieves the minimum maximum drawdown.

The remainder of this paper is organized as follows: Section 2 discusses related work; Section 3 formally defines the research problem; Section 4 details the proposed EI^3 network, as well as the recurrent reinforcement learning framework based on EI^3; Section 5 presents and analyzes the experimental results; Finally, Section 6 concludes this paper.

2 RELATED WORK

Currently, most machine learning-based methods in finance domain focus on supervised learning. There are linear models like the well-known autoregressive integrated moving average (ARIMA) model [4], which cannot be used to model nonlinear relationships. Nonlinear models, such as recurrent neural network (RNN) [31], long short-term memory (LSTM) units [18], gated recurrent unit (GRU) [9] and encoder-decoder [9], are designed for temporal sequence processing, mainly for price or trend prediction. However, prediction is difficult in non-stationary stock market and prediction accuracy is not enough in making trading decisions, because some important constraints like trading costs are difficult to import in price prediction stage. In addition, after making prediction, trading managers need to decide the trading actions themselves, which is not straightforward and moreover, manual operation sometimes may lead to failure in making profit.

Reinforcement learning in finance directly making trading decisions according to the recent price movement without prediction. There are three categories of research about reinforcement learning in trading market: (a) critic-only methods [10] that learn a value function to evaluate ("criticize") the action benefits; (b) actor-only methods (also called direct reinforcement learning) [12] that directly decide trading action according to recent price and the last action; and (c) actor-critic methods [5] that use an actor to produce trading action and a critic to evaluate the action performance. However, all these methods are not suitable for portfolio management. One important limitation is that these methods consider only trading single asset and do not involve the allocation among multiple assets. Another limitation is that the action space of these researches is discrete. For portfolio management involving multiple assets, discrete action for one asset will lead to an exponential growth of the action space [13].

To solve the portfolio management problem, Jiang *et al.* [21] proposed a deterministic deep reinforcement learning method with a network consisting of one constitutional layer and two fully connected layer, within which all the assets are considered in union to decide the allocation weights among multiple assets. In [22], Jiang *et al.* further proposed an ensemble of identical independent evaluators (EIIE). The most important characteristic of EIIE is that data flows independently for each asset while parameters are shared. There are three implementations of EIIE: CNN EIIE, basic RNN EIIE and LSTM EIIE. CNN EIIE uses a CNN module to extract price movement information and the first dimension (asset dimension) of convolution kernel is set to 1 to guarantee that the time series of each asset are processed independently. Basic RNN EIIE and LSTM EIIE perform recurrent learning on each asset, and different assets share the same network parameters. Though reinforcement learning for portfolio management has made great progress, these methods did not take full use of time series to capture temporal information, which makes their performance still not satisfactory.

On the other line, by now the development of CNN has made great progress in computer vision. Researchers have

proposed various CNNs, such as VGG [34], Inception [35] and DenseNet [20], to promote the accuracy of image classification. However, though advanced CNN architectures have played an important role in the area of computer vision, they are not fully researched for time series analysis, let alone reinforcement learning for portfolio management. In this paper, we build this gap by borrowing the idea of Inception to solve the multiple-scale temporal feature extraction problem, with the objective of capturing more information, and eventually, providing a better solution to the portfolio management problem.

3 PROBLEM DEFINITION

Portfolio management is a process of continuously redeploying funds into different assets. Assume a trader is in a financial market where price ticks with opening, highest, lowest and closing prices arrive at a fixed period, given a portfolio with m assets and an initial cash, the trader considers how to rebalance portfolio periodically, with the objective of maximizing the total profit in a given time interval. Specifically, at the beginning of each period, trader observes current price movement and allocates the fund into these assets. Then price of each asset changes during the period and the portfolio value changes accordingly. At the end of the period, trader receives the profit and reallocates the fund to the assets. The financial market is assumed to be continuous, which means the opening price of period t is actually the closing price of the previous period $t-1$. Therefore, the opening price information is not used in this work. At the end of period t, price vector of the portfolio can be defined as,

$$\mathbf{v}_t = [v_{0,t}, v_{1,t}, \cdots, v_{m,t}]^\mathsf{T} \tag{1}$$

where $v_{i,t}$ $(1 \le i \le m)$ is the closing price of the i-th asset at period t, and $v_{0,t}$ denotes the cash (Bitcoin in this paper) and its price is always 1. The highest and lowest price vectors of period t, denoted by $\mathbf{v}_t^{(\text{hi})}$ and $\mathbf{v}_t^{(\text{lo})}$, respectively, can be defined in a similar way as $\mathbf{v}_t$. During period t, the price change can be defined as *price relative vector*,

$$\mathbf{y}_t = \mathbf{v}_t \oslash \mathbf{v}_{t-1} = \left[1, \frac{v_{1,t}}{v_{1,t-1}}, \frac{v_{2,t}}{v_{2,t-1}}, \cdots, \frac{v_{m,t}}{v_{m,t-1}}\right]^\mathsf{T} \tag{2}$$

where $\oslash$ denotes the element-wise division operator.

Trader reallocates fund into different assets according to the portfolio vector, which is represented as,

$$\mathbf{w}_t = [w_{0,t}, w_{1,t}, \cdots, w_{m,t}]^\mathsf{T} \tag{3}$$

where $w_{i,t}$ denotes the weight of the i-th asset at period t. Note that for any period t, there is $\sum_{i=0}^{m} w_{i,t} = 1$. Moreover, we let $\mathbf{w}_0 = [1, 0, \cdots, 0]^\mathsf{T}$, indicating that we have only cash before investment. The portfolio value can be computed as,

$$p_t = \mu_t p_{t-1} \mathbf{y}_t \cdot \mathbf{w}_{t-1} \tag{4}$$

where μ_t is the transaction remainder factor, which represents portfolio value shrinking degree after paying commission fees. Detail description of μ_t can be found in [22]. To prevent the effect of initial values, we set $p_0 = 1$.

In order to derive the final accumulated profit, the profit of period t is modeled as the logarithmic rate of return,

$$r_t = \ln \frac{p_t}{p_{t-1}} = \ln(\mu_t \mathbf{y}_t \cdot \mathbf{w}_{t-1}) \tag{5}$$

Consequently, the final portfolio value, denoted by $\mathcal{P}$, can be computed by,

$$\mathcal{P} = p_0 \exp\left(\sum_{t=1}^{T+1} r_t\right) \tag{6}$$

Note here T is the last allocation period in portfolio management process and r_{T+1} is the profit after the last allocation.

Based on the above description, the portfolio management problem to be addressed in this work is to determine a series of $\mathbf{w}_t$ $(1 \le t \le T)$, so that $\mathcal{P}$ can be maximized.

4 METHOD

Since we use reinforcement learning to solve the portfolio management problem, we first formally define notations and the portfolio management problem via reinforcement learning in Section 4.1, and then detail the proposed EI^3 network in Section 4.2, finally, we describe the recurrent reinforcement learning framework based on EI^3 in Section 4.3.

4.1 Reinforcement Learning for Portfolio Management

The portfolio management problem can be modeled as a Markov Decision Process (MDP), which assumes the current state only depends on the previous state and action. Formally, an MDP is a five-tuple $< S, A, P, R, \gamma >$, where S is a set of states, A is a set of actions, $P : S \times A \times S \to \mathbb{R}$ is transition probability distribution, $R : S \times A \to \mathbb{R}$ is reward function, and $\gamma \in [0, 1]$ is the discount factor. Detailed descriptions of MDP for portfolio management are as follows:

- **State**. When deciding the trading action at period t, a stocker trader will consider both the recent price movement in time window n (n is user defined), denoted by $\mathbf{X}_t$, and the previous portfolio weight vector $\mathbf{w}_{t-1}$, i.e.,

$$s_t = (\mathbf{X}_t, \mathbf{w}_{t-1}) \tag{7}$$

  Note here $\mathbf{X}_t$ is a price tensor, which is the stacking of the closing, highest and lowest normalized price matrices, represented respectively by,

$$\begin{aligned}
\mathbf{V}_t &= [\mathbf{v}_{t-n+1} \oslash \mathbf{v}_t | \mathbf{v}_{t-n+2} \oslash \mathbf{v}_t | \cdots | \mathbf{v}_{t-1} \oslash \mathbf{v}_t | \mathbf{1}] \\
\mathbf{V}_t^{(\text{hi})} &= \left[\mathbf{v}_{t-n+1}^{(\text{hi})} \oslash \mathbf{v}_t^{(\text{hi})} | \mathbf{v}_{t-n+2}^{(\text{hi})} \oslash \mathbf{v}_t^{(\text{hi})} | \cdots | \mathbf{v}_{t-1}^{(\text{hi})} \oslash \mathbf{v}_t^{(\text{hi})} | \mathbf{1}\right] \\
\mathbf{V}_t^{(\text{lo})} &= \left[\mathbf{v}_{t-n+1}^{(\text{lo})} \oslash \mathbf{v}_t^{(\text{lo})} | \mathbf{v}_{t-n+2}^{(\text{lo})} \oslash \mathbf{v}_t^{(\text{lo})} | \cdots | \mathbf{v}_{t-1}^{(\text{lo})} \oslash \mathbf{v}_t^{(\text{lo})} | \mathbf{1}\right]
\end{aligned} \tag{8}$$

  with $\mathbf{1} = [1, 1, \ldots, 1]^\mathsf{T}$.
- **Action**. In portfolio management, the action at period t is to decide the weight vector $\mathbf{w}_t$, so,

$$a_t = \mathbf{w}_t \tag{9}$$

- **Reward**. The immediate reward is the profit of period t, i.e., r_t.

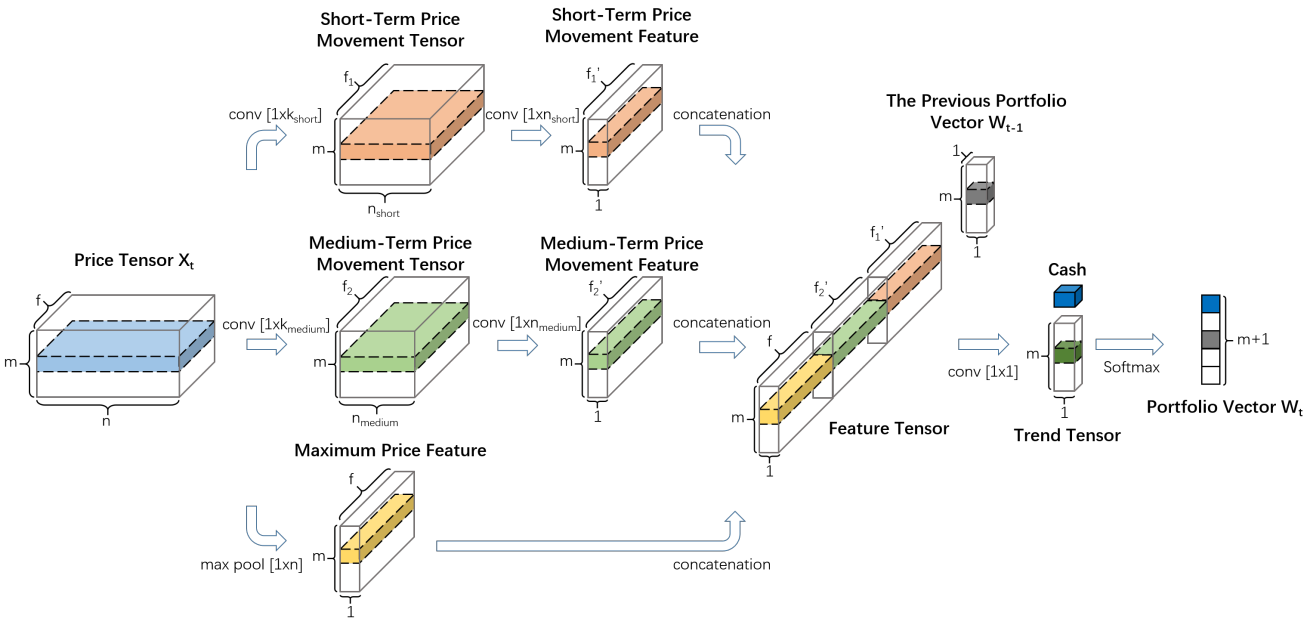


Figure 1: The architecture of our EI^3 network.

The goal of reinforcement learning is to find an optimal policy $\pi : S \to A$ that maximizes the accumulated reward.

For a time interval $[t_a, t_b]$, the accumulated reward is defined as the average logarithmic cumulated return,

$$R(s_{t_a}, a_{t_a}, \ldots, s_{t_b}, a_{t_b}, s_{t_b+1}) = \frac{1}{t_b + 1 - t_a} \sum_{t=t_a}^{t_b+1} r_t \quad (10)$$

Trading agent takes actions (rebalances portfolio weights), receives feedback (reward) from the environment, and updates trading policy to increase accumulated reward. We use deterministic policy gradient [33] to find the optimal policy that maximize the objective function $J_{[t_a,t_b]}(\pi_\theta)$, where θ is a set of parameters, and π_θ is the policy parameterized by θ. In this work, we use the reward in a time interval $[t_a, t_b]$, i.e., $R(s_{t_a}, a_{t_a}, \ldots, s_{t_b}, a_{t_b}, s_{t_b+1})$ as the objective function, but it is worth mentioning that other performance measures such as Sharpe ratio [29], expected maximum drawdown [2], can also be used as the objective functions. The parameters are updated in a mini-batch within time range $[t_a, t_b]$ via gradient ascent according to the following formula,

$$\theta \longrightarrow \theta + \lambda \nabla_\theta J_{[t_a,t_b]}(\pi_\theta) \quad (11)$$

where λ is the learning rate.

4.2 EI^3 Network

4.2.1 Overview. Figure 1 depicts the architecture of the proposed EI^3 network, which consists of three modules: a multi-scale temporal feature extraction module that has three branches, an integration module and a normalization module. The input of the network is price tensor $\mathbf{X}_t$ and the previous portfolio vector $\mathbf{w}_{t-1}$. The price tensor will run through the three branches, then integrate with the previous portfolio vector $\mathbf{w}_{t-1}$, and finally normalize to get the current action at period t. The three branches are used to extract multi-scale features simultaneously. Specifically, the first branch is the short-term price movement branch that consists of two convolution layers, where the first one is to generate short-term price movement tensor using a small kernel size, and the second one is to compress short-term price movement tensor into a feature tensor with width 1. The second branch is the medium-term price movement branch, which also consists of two convolution layers that work similarly as that in the first branch. Note that the kernel size of the first layer of the second branch is larger than that in the first branch, which makes its receptive fields larger, and thus can capture temporal information within a longer period. The third branch is to select the maximum closing (highest, lowest) price in the time window n. Then, the integration module is responsible for aggregating these features with the previous portfolio vector to generate a comprehensive trend tensor. Finally, these m features of assets, combining with 1 cash (Bitcoin), are normalized to get the portfolio vector $\mathbf{w}_t$ (also the action at period t).

Note that $\mathbf{X}_t$ is a tensor with form (f, n, m), where $f = 3$ is the feature number (closing, highest, lowest), n is the time window size, and m is the asset number. In computer vision, an image tensor has three dimensions, the channel dimension, the width dimension and the height dimension. The first dimension of the price tensor is feature dimension, corresponding to the channel dimension in image tensor. The second dimension is temporal dimension, corresponding to the width dimension in image tensor, and the third dimension is asset dimension, corresponding to the height dimension in image tensor. Note that in computer vision, a convolution layer needs to learns cross-channel correlations and spatial correlations in a 3D space. However, in portfolio management,

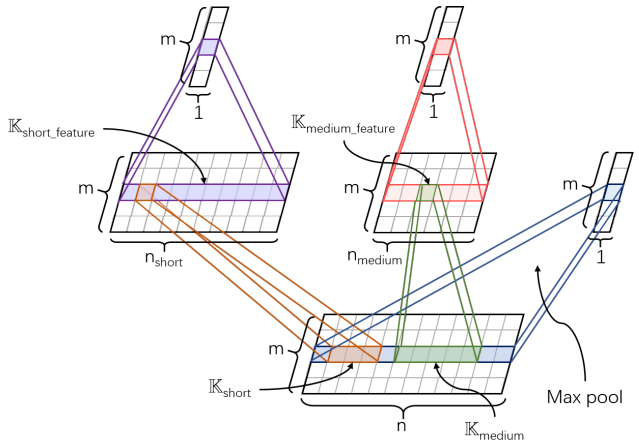


Figure 2: The architecture of multi-scale temporal feature extraction module.

a convolution layer only needs to learn cross-feature (channel) correlations and temporal (width) correlations. This is because assets are disordered, which means there is actually no logical connection between adjacent assets on the tensor. Moreover, channels of image (RGB) are homogeneous while features of price tensor are heterogeneous. Therefore, it is meaningless to consider the maximum, average or other statistical characteristics between different features. To keep assets independent, the same as in EIIE [22], we set the first dimensions of all the local receptive fields in all feature maps to 1. This operation guarantees that the process of assets isolated from each other until the final normalization. Therefore, the network flows independently for the m assets while network parameters are shared among these streams. Each asset is evaluated respectively, and then a price movement feature tensor can be extracted. Combining with the previous portfolio vector $\mathbf{w}_{t-1}$, the network outputs a tensor representing the asset investment preference degree. Afterwards, the cash tensor is added, and the Softmax function is applied to allocate weights among the $m+1$ assets. Finally, we can get $\mathbf{w}_t$.

4.2.2 Multi-Scale Feature Extraction. The temporal feature of each asset will be extracted by the feature extraction module, which is shown in Figure 2. This module consists of three branches that extract short-term price movement feature, medium-term price movement feature, and maximum price feature respectively. Convolution with different kernel sizes will perform through each assets first, and then the temporal dimension of feature maps will be limited to 1 for later integration.

Short-term price movement feature. In financial technical analysis, short-term price movement is used to model microcosmic fluctuation of asset value. We first apply convolution with filter $\mathbb{K}_{\text{short}} = [1 \times k_{\text{short}}]$ on price tensor $\mathbf{X}_t$ for fluctuation extraction and noise reduction within time window k_{short}. Feature map for each channel of short-term price movement tensor is computed as,

$$\mathbf{X}_{\text{short}} = \varphi_{\text{short}} \left([\mathbf{X}_t * \omega_{\text{short}}] + b_{\text{short}} \right) \tag{12}$$

where φ_{short} is the activation function, ω_{short} and b_{short} are weights and bias of the convolution layer, $*$ denotes the convolution operator. For the activation function, we use ReLU throughout this paper if not otherwise specified.

After processed by the feature extraction layer, a tensor with shape $(f_1, n_{\text{short}}, m)$ is generated to represent the short-term fluctuation information, where f_1 is a user defined feature number and $n_{\text{short}} = n - k_{\text{short}} + 1$ is the time window number. For later integration, convolution with filter $\mathbb{K}_{\text{short_feature}} = [1 \times n_{\text{short}}]$ is used to compress this tensor, so the time window dimension is decreased from n_{short} to 1,

$$\mathbf{X}'_{\text{short}} = \varphi'_{\text{short}} \left([\mathbf{X}_{\text{short}} * \omega'_{\text{short}}] + b'_{\text{short}} \right) \tag{13}$$

where φ'_{short} is the activation function, ω'_{short} and b'_{short} are weights and bias of the convolution layer. In this step, we can get the short-term movement feature with shape $(f'_1, 1, m)$, where f'_1 is a user defined feature number.

Medium-term price movement feature. Besides short-term fluctuation, temporal information within longer term period is important for evaluating medium-term fluctuations and trends. Therefore, convolution with filter $\mathbb{K}_{\text{mid}} = [1 \times k_{\text{mid}}]$ is applied to derive the medium-term price feature tensor. The feature map for each channel is computed by,

$$\mathbf{X}_{\text{medium}} = \varphi_{\text{medium}} \left([\mathbf{X}_t * \omega_{\text{medium}}] + b_{\text{medium}} \right) \tag{14}$$

where φ_{medium} is the activation function, ω_{medium} and b_{medium} are weights and bias of the convolution layer. Here we can get the medium-term price movement tensor with shape $(f_2, n_{\text{medium}}, m)$, where f_2 is a user defined feature number and $n_{\text{medium}} = w - k_{\text{medium}} + 1$ is the time window number. The same as in short-term price movement feature extraction, convolution with filter $\mathbb{K}_{\text{medium_feature}} = [1 \times n_{\text{medium}}]$ is used to compress the derived tensor,

$$\mathbf{X}'_{\text{medium}} = \varphi'_{\text{medium}} \left([\mathbf{X}_{\text{medium}} * \omega'_{\text{medium}}] + b'_{\text{medium}} \right) \tag{15}$$

where φ'_{medium} is the activation function, ω'_{medium} and b'_{medium} are weights and bias of the convolution layer. In this set, we can get the medium-term price movement feature with shape $(f'_2, 1, m)$, where f'_2 is a user defined feature number.

Maximum price feature. For financial technical analysis, the highest price during a given interval is one of the most important basic indicators. Hence, we use max pooling to find the maximum closing, highest and lowest price, respectively,

$$\mathbf{X}_{\max} = \text{maxpool} \left(\mathbf{X}_t \right) \tag{16}$$

Here we can get the maximum price feature tensor with shape $(f, 1, m)$.

4.2.3 Integration. In this module, we concatenate the short-term, medium-term price movement features and the maximum price feature to get the feature tensor with shape $(f'_1 + f'_2 + f, 1, m)$. The feature tensor is a container for all information about the price movement of an asset, by taking all the factors into consideration. The profit of trading agent depends on not only the price movement, but also

the previous trading decision. Therefore, in addition to the current price movement, the previous portfolio vector should also be included, because it affects the trading costs when making decisions. So, we can get a tensor $\mathbf{X}_{\text{concat}}$ with shape $(f_1' + f_2' + f + 1, 1, m)$. Note that reinforcement learning methods that depend on both observations and the last action is called recurrent reinforcement learning, so our method can be regarded as a recurrent reinforcement learning method.

After concatenation, in order to get the trend vector, features should be compressed into 1 channel. We use a 1×1 convolution to perform cross-channel feature aggregation, since 1×1 convolution can add more nonlinearity (through the activation function) into network, compared with fully connected layer. The feature map is then computed by,

$$\mathbf{X}_{\text{trend}} = \varphi_{\text{trend}} \left([\mathbf{X}_{\text{concat}} * \omega_{\text{trend}}] + b_{\text{trend}} \right) \tag{17}$$

where φ_{trend} is the activation function, ω_{trend} and b_{trend} are weights and bias of the convolution layer.

4.2.4 Normalization. By now we have extracted the price movement information of each asset. We assume the price of cash is invariable and there is no need to consider its price movement information. However, when considering how to rebalance among multiple assets, cash should also be included, because it is important for hedging in bear market while keeping stable in bull market. We use the Softmax function to normalize the $m + 1$ assets. Formally,

$$\mathbf{w}_t = [w_{0,t}, w_{1,t}, \cdots, w_{m,t}]^{\mathsf{T}} \tag{18}$$

and

$$w_{i,t} = \frac{\exp \mathbf{v}_{\text{trend}}(i)}{\sum_{i=0}^{m} \exp \mathbf{v}_{\text{trend}}(i)} \tag{19}$$

where $\mathbf{v}_{\text{trend}}$ is a vector with shape $m + 1$, squeezed from $\mathbf{X}_{\text{trend}}$ concatenated with the cash tensor.

4.3 Framework

The detail of our recurrent reinforcement learning framework for portfolio management based on EI^3 is shown in Algorithm 1. The same as [22], we use portfolio-vector memory (PVM) to store portfolio vectors in chronological order for training.

We first initialize network parameters and PVM (Line 1), then in each step, we randomly sample prices in continuous periods from the database (Line 3) and select corresponding portfolio vector from PVM (Line 4) for training. For each price data in the mini-batch, we select prices within time window n and normalize them to obtain price relative vector $\mathbf{X}_t$ (Lines 6-7). Afterwards, we input the price relative vector and previous portfolio vector $\mathbf{w}_{t-1}$ into the EI^3 network, then get the next portfolio vector $\mathbf{w}_t$ (Line 8) and store it in PVM (Line 9). Trading agent re-balances portfolio weights into vector by $a_t = \mathbf{w}_t$ (Line 10). Due to price movement during period t, portfolio value changes and we can get the corresponding profits (Line 11). After going through the whole mini-batch, we obtain the average logarithmic accumulated reward as the objective function of policy gradient (Line 13), and update network parameters via the deterministic policy gradient algorithm (Line 14).

Algorithm 1: Recurrent Reinforcement Learning based on EI^3 Network

```
1  Initialize EI^3 network and PVM;
2  for step = 1 ... L do
3      Sample a random mini-batch of prices (with range
       [t_a, t_b]) in continuous periods from the database;
4      Select corresponding portfolio vectors from PVM;
5      for t = t_a ... t_b do
6          Select closing, highest, lowest prices
           [v_{i,t}, v_{i,t}^(hi), v_{i,t}^(lo)] within time window n;
7          Normalize the prices by (8) and stack them as
           price vector X_t;
8          Input state s_t = (X_t, w_{t-1}) into network and get
           the current portfolio vector w_t;
9          Update portfolio vector w_t in PVM;
10         Take action by a_t = w_t;
11         Compute current portfolio value p_t by (4) and
           reward r_t by (5);
12     end
13     Compute average accumulated reward by (10);
14     Update network parameters by (11);
15 end
```

5 EXPERIMENTS

We first describe the experimental setup in Section 5.1, and then present and discuss the results in Section 5.2.

5.1 Experimental Setup

5.1.1 Data Preparation and Hyper-parameter Setting. Our price datasets are obtained from Poloniex through the official application programming interface (https://poloniex.com/support/api/). We collect prices of the top $m = 11$ volumed cryptocurrencies (called coins for convenience), as depicted in Figures 3-5, and their total periods, as shown in Table 1. We divide the 2 year time range into two parts. The first part includes the first 22 months and is utilized as data training period (left side of the black line in the three figures). The second part that includes the last 2 months is testing period (right side of the black line in the three figures).

Figures 3-5 present the price variations of the three back-tests. It can be observed that the financial market is highly volatile and non-stationary. Prices of different coins differ greatly and the price of a coin may rise and fall rapidly. In addition, some coins appear only recently, as suggested in [22], we assume their prices keep unchanged in the missing

Table 1: Price data ranges for three back-tests.

Dataset	Training Period	Testing Period
Back-Test 1	2015-07-01 to 2017-05-03	2017-05-03 to 2017-07-01
Back-Test 2	2016-01-01 to 2017-11-03	2017-11-03 to 2018-01-01
Back-Test 3	2016-07-01 to 2018-05-03	2018-05-03 to 2018-07-01

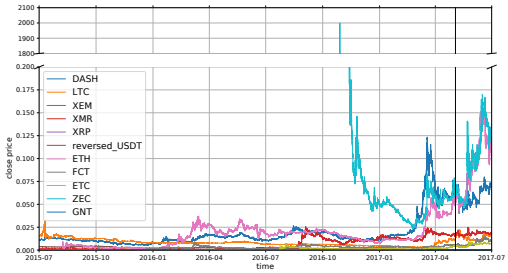


Figure 3: Price variation of Back-Test 1

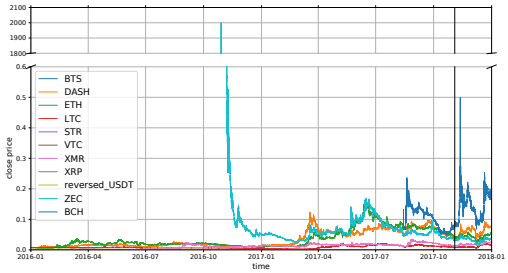


Figure 4: Price variation of Back-Test 2

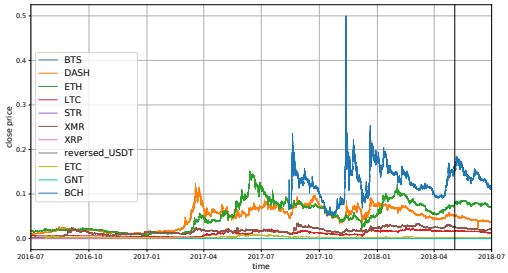


Figure 5: Price variation of Back-Test 3

periods. The three testing datasets (see right side of the black line in Figures 3-5) present different price change patterns - upswing, smooth fluctuation, and downswing. In Back-Test 1 (Figure 3), prices of most assets keep rising or falling moderately, and it can be seen as a bull market. In Back-Test 2 (Figure 4), prices meet oscillation and rise and fall repeatedly, and the trend can be viewed as smooth fluctuation. In Back-Test 3 (Figure 5), prices keep falling and it can be regarded as a bear market.

For the size of the filters in the multi-scale feature extraction module, we set $k_{\text{short}} = 3$, $k_{\text{mid}} = 21$, $f_1 = f_2 = 3$ and $f_1' = f_2' = 20$. We do not involve in portfolio selection issues in this work, so we simply take top $m = 11$ currencies in volume over the last 30 days in the training data sets. We perform portfolio weight adjustment every 30 minutes, i.e., the period is set to 30 minutes. Time window n is set to 50 (one day and an hour) and trading fee ratio is set to 0.25% (the maximum rate at Poloniex). In training process, the learning rate is 0.001 and decays 0.9 every 1000 steps. The Adam algorithm is utilized for learning rate optimization. For network parameters, we use the xavier uniform initializer [15] to initialize them.

5.1.2 Baselines. We compare our model with several state-of-the-art deep reinforcement learning based portfolio management methods to verify the effectiveness of the proposed EI^3 based method. For convenience, we use EI^3 to denote our method in the following discussion.

- **CNN EIIE (CNN)** [22]. This method uses a CNN module to extract price movement information and sets the first dimension of convolution kernel to 1.
- **basic RNN EIIE (RNN)** [22]. For each asset, a basic RNN will run through the time window, and all the assets share the same subnet RNN.
- **LSTM EIIE (LSTM)** [22]. For each asset, a LSTM will run through the time window, and all the assets share the same subnet LSTM.
- **Integrated CNN(iCNN)** [21]. This method uses the historic prices of a set of assets as input and outputs the portfolio weights. In iCNN, all the assets are considered in union.

Besides machine learning based approaches, there are also plenty of traditional portfolio management approaches, as summarized in [25]. For a comprehensive comparison, we also include some of them as competitors.

- Benchmarks. **Uniform Buy and Hold (UBAH)** that invests in the assets equally and holds them during the whole interval; **Best Stock** that holds the asset with the maximum final accumulative portfolio value over the test datasets (note that **Best stock** is a hindsight strategy); **Uniform Constant Rebalanced Portfolios (UCRP)** that rebalances the portfolio to a uniform portfolio every period.
- Follow-the-winner approaches. Increases the weights of more profitable assets, such as **Universal Portfolios (UP)**[11], **Exponential Gradient (EG)**[17], **Online Newton Step (ONS)**[1].
- Follow-the-loser approaches. Transfer the weights of more profitable assets into less profitable assets due to mean reversion, such as **Anticor**[7], **Passive Aggressive Mean Reversion (PAMR)**[26], **Online Moving Average Reversion (OLMAR)**[23], **Weighted Moving Average Mean Reversion (WMAMR)**[14], **Robust Median Reversion (RMR)**[19].
- Pattern-matching based approaches. First select similar samples from historical price movements and then construct the optimal portfolio according to the similar set, such as **M0** [6], **Nonparametric Kernel Based Log Optimal Strategy (B^{K})** [16], **Correlation-driven Nonparametric Learning (CORN)** [24].

5.1.3 Performance Measures. We use commonly used measures, including final accumulative portfolio value, sharpe ratio and maximum drawdown, to evaluate the performance of the tested methods.

final Accumulative Portfolio Value (fAPV). The most important objective of portfolio management is to make

Table 2: Performance of evaluated portfolio management methods in testing dataset under three back-tests.

	Back-Test 1			Back-Test 2			Back-Test 3		
	fAPV	SR	MDD	fAPV	SR	MDD	fAPV	SR	MDD
EI^3	**93.1247**	**0.0916**	0.2482	**15.8954**	**0.0619**	**0.3110**	0.9744	-0.0358	**0.0303**
CNN[22]	51.2635	0.0848	0.2400	9.0914	0.0514	0.4929	0.3452	-0.0603	0.6648
RNN[22]	34.5256	0.0841	0.2975	6.2007	0.0485	0.4307	0.8194	-0.0252	0.2054
LSTM[22]	55.2366	0.0851	0.2405	15.8863	0.0579	0.3919	0.8170	-0.0240	0.2050
iCNN[21]	1.1679	0.0128	0.5110	1.6873	0.0207	0.4535	0.7909	-0.0148	0.3591
UBAH	1.4944	0.0211	0.2678	2.5571	0.0383	0.4809	0.8501	-0.0243	0.1699
Best Stock	2.3168	0.0249	0.6085	6.3607	0.0453	0.4137	**1.5214**	**0.0339**	0.1053
UCRP	1.7750	0.0326	0.2336	2.3552	0.0389	0.4331	0.8324	-0.0253	0.1917
UP[11]	1.7541	0.0318	0.2348	2.3805	0.0390	0.4342	0.8335	-0.0253	0.1900
EG[17]	1.7624	0.0321	0.2345	2.3700	0.0390	0.4337	0.8332	-0.0252	0.1906
ONS[1]	1.7563	0.0323	0.2200	1.3975	0.0171	0.5091	0.7847	-0.0222	0.2666
Anticor[7]	4.9882	0.0550	**0.1871**	2.1449	0.0294	0.5251	0.3604	-0.0947	0.6410
PAMR[26]	0.0458	-0.0461	0.9780	0.0094	-0.0720	0.9911	0.0002	-0.4400	0.9998
OLMAR[23]	4.5319	0.0365	0.5936	0.4740	-0.0022	0.7512	0.0027	-0.2928	0.9973
WMAMR[14]	1.8537	0.0212	0.5468	0.9004	0.0078	0.7001	0.0408	-0.1776	0.9594
RMR[19]	7.9878	0.0470	0.5724	0.4898	-0.0012	0.7379	0.0037	-0.2757	0.9963
M0[6]	1.9071	0.0287	0.2876	2.5194	0.0364	0.4396	0.7789	-0.0288	0.2410
B^K[16]	0.5968	-0.0022	0.7797	0.7281	0.0049	0.8107	1.1217	0.0107	0.2312
CORN[24]	0.0274	-0.0828	0.9767	0.0039	-0.0980	0.9961	0.0010	-0.5673	0.9990

profits. Therefore, the final accumulative portfolio value reflects the performance of portfolio management strategies well, and is computed by,

$$fAPV = \frac{\mathcal{P}}{p_0} \tag{20}$$

where $\mathcal{P}$ is the final portfolio value and p_0 is the initial value, which is assumed to be 1 unless otherwise specified.

Sharpe Ratio (SR) [32]. Though fAPV evaluates the profit, it does not take risk into account. The Sharpe Ratio is a risk adjusted mean return, defined as the average of the risk-free return by its deviation,

$$SR = \frac{\mathbb{E}_t[\rho_t - \rho_F]}{\sqrt{\mathrm{var}_t[\rho_t - \rho_F]}} \tag{21}$$

where $\rho_t = p_t/p_{t-1}$ is the rate of return, ρ_F is the rate of return of risk-free asset and usually is assumed to be 0, if not otherwise specified.

Maximum DrawDown (MDD) [27]. In Sharpe ratio, risk is modeled as the volatility of portfolio value, but traders usually take downward movements instead of upward movements as risk. MDD is the maximum drawdown over the whole testing set, which is defined as,

$$MDD = \max_{\tau} \left(\max_{t<\tau} \frac{p_t - p_\tau}{p_t} \right) \tag{22}$$

Note that fAPV and SR are measures for return and risk adjusted return, so higher fAPV and SR means higher profitability of trading strategies. MDD is a measure for risk, lower MDD means less loss chances.

5.2 Results

5.2.1 Overall Performance. The performances of all the evaluated methods in the three testing datasets are summarized in Table 2, where the best performance is in boldface. It can be observed that there is no absolutely best method that outstands in all the three back-tests. However, our method EI^3 achieves the highest fAPV and SR in both Back-Test 1 and Back-Test 2, the third highest fAPV in Back-Test 3, and the lowest MDD in both Back-Test 2 and Back-Test 3, which demonstrates the profitability and reliability of our method on solving the portfolio management problem. In upswing environment (Back-Test 1), the final wealth of EI^3 is as high as 93.1247, while the highest number of other deep learning methods is 55.2366 by LSTM, and the highest number of traditional strategies is 7.9878 by RMR. Due to its high portfolio value, EI^3 also achieves the highest SR 0.0916. Though frequent trading leads to higher MDD (higher risk), it is still acceptable, especially when considering the high profit. In smooth fluctuation environment (Back-Test 2), EI^3 also obtains the highest final wealth 15.8954, while the highest number of other deep learning methods is 15.8863 by LSTM, and the highest number of traditional strategies is 6.36 by Best Stock. Note that although the fAPV of LSTM is close to that of EI^3, our method exhibits higher SR and lower MDD, which indicate that our method can achieve higher profit with lower risk. In downswing environment (Back-Test 3), only hindsight strategy Best Stock and B^K make profits (with fAPV larger than 1). In fact, in the decline stage, minimizing losses is more important than making profits.

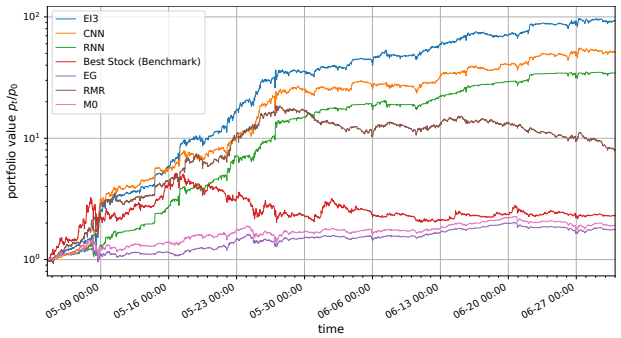


Figure 6: Accumulated portfolio values over the testing interval of Back-Test 1.

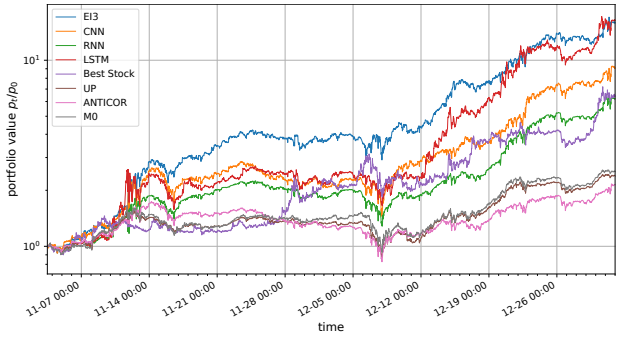


Figure 7: Accumulated portfolio values over the testing interval of Back-Test 2.

The fAPV of EI^3 is approximately equal to 1 and MDD also approximates to 0, because EI^3 tends to hold risk-free asset Bitcoin instead of risky assets to avoid loss during the decline stage. In general, EI^3 can make higher profit in bull market and stay steady in bear market. The reason is that the multi-scale temporal information extracted in EI^3 provides more profitable opportunities and loss chances at multiple granularities.

It can also be observed that other deep reinforcement learning methods (except iCNN) perform better than traditional strategies in Back-Test 1 and Back-Test 2, this demonstrates that deep reinforcement learning methods have great potential in solving the portfolio management problem. Moreover, among traditional strategies, though benchmark methods are simple, only a few algorithms can beat them. This reflects the difficulty in portfolio management and the importance of considering transaction costs when making trading decisions.

5.2.2 Performance in Different Market Trends. We also evaluate the performance of the accumulated portfolio values over the testing interval of the three back tests. In this set of experiments, we compare EI^3 with CNN EIIE, basic RNN EIIE, LSTM EIIE, and other four methods that each achieves the best performance in the four categories.

Upswing Environment. Figure 6 shows accumulated portfolio values of EI^3, CNN, RNN, LSTM, Best Stock, EG, RMR and M0 over the testing interval of Back-Test 1. As a benchmark, portfolio value of Best Stock increases to 1.5

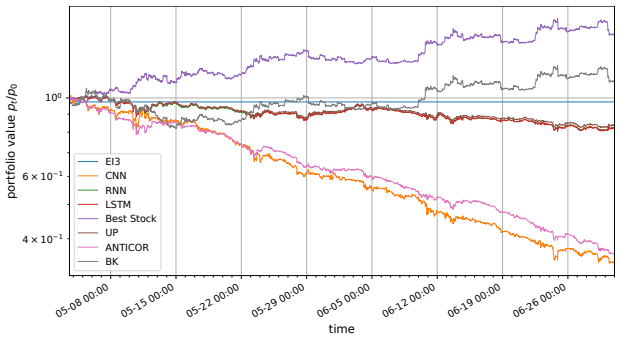


Figure 8: Accumulated portfolio values over the testing interval of Back-Test 3.

before 05-17, then decreases to 1.1 and stays steady. The performance of other algorithms keep increasing when Best Stock starts to decrease, this is because other assets make profit while Best Stock suffers a loss. For traditional strategies, portfolio values of M0 and EG increase slowly, while that of RMR rises at the beginning and then drops since 05-27. All the deep reinforcement learning methods perform outstandingly, whereas EI^3 exhibits the best performance among them.

Smooth Fluctuation Environment. Figure 7 shows the accumulated portfolio values of EI^3, CNN, RNN, LSTM, Best Stock, UP, Anticor and M0 over the testing interval of Back-Test 2. We can see that all the methods exhibit similar performance, except Best Stock, whose value increases at first and keeps steady since 11-25, and then increases again after temporary fluctuation at 12-08. EI^3 again exhibits the dominate performance, as compared to other methods. Note that although the final portfolio value of LSTM is close to that of EI^3, its portfolio value is, for the most part, lower than that of EI^3. This demonstrates the profitability and reliability of EI^3.

Downswing Environment. Figure 8 shows the accumulated portfolio values of EI^3, CNN, RNN, LSTM, Best Stock, UP, Anticor and B^{K} over the testing interval of Back-Test 3. In downswing environment, only Best Stock keeps making profit through the whole interval. However, Best Stock is a hindsight strategy and cannot realize in reality. B^{K} wanders between making profit and suffering a loss at the beginning, but then continues to profit after 06-10 and finally achieves the second highest value. CNN and Anticor suffer great losses, while RNN and UP suffer little. Different from all these methods, portfolio value of EI^3 keeps to be approximately to 1, because it devotes almost all the capital into risk-free cash (Bitcoin). This allocation strategy is owe to the comprehensive analysis of multi-scale temporal information.

From the experiments, we can conclude that there is no best strategy that can stand out in all the environments. However, EI^3 can make the highest profit in upswing and smooth fluctuation environments, while prevent losses in downswing environment, due to that multi-scale temporal information extracted in EI^3 helps capture profitable opportunity and loss chances at various granularities.

6 CONCLUSION

In this paper, we studied the financial portfolio management problem. Inspired by the Inception network in computer vision, we proposed a novel Ensemble of Identical Independent Inception (EI^3) convolutional neural network, which is capable of considering multi-scale temporal information for comprehensive evaluation of profitability and loss. Three branches in EI^3 were designed to extract short-term, medium-term and maximum temporal information respectively. The extracted features were then integrated with previous portfolio weights to generate the trend information, and finally, a normalization process was conducted on the trend information to obtain the next portfolio weights. Based on EI^3, we proposed a recurrent reinforcement learning framework to solve the portfolio management problem. Empirical study on three back-tests with a trading period of 30 minutes in a cryptocurrency market show that, compared to existing solutions, our method can obtain higher profits in bull market and lower losses in bear market. Our attempt in this work reveals the importance of considering temporal information at multiple scales when making trading decisions.

ACKNOWLEDGMENTS

The work was partially supported by the National Natural Science Foundation of China under Grant No. 61672252, and the Fundamental Research Funds for the Central Universities under Grant No. 2019kfyXKJC021.

REFERENCES

[1] Amit Agarwal, Elad Hazan, Satyen Kale, and Robert E. Schapire. 2006. Algorithms for portfolio management based on the Newton method. In *Proceedings of ICML*. 9–16.

[2] Saud Almahdi and Steve Y. Yang. 2017. An adaptive portfolio trading system: A risk-return portfolio optimization using recurrent reinforcement learning with expected maximum drawdown. *Expert Systems with Applications* 87 (2017), 267–279.

[3] Gerald Appel. 2005. *Technical analysis: power tools for active investors*.

[4] Adebiyi Ariyo Ariyo, Aderemi Oluyinka Adewumi, and Charles K. Ayo. 2014. Stock Price Prediction Using the ARIMA Model. In *Proceedings of UKSim*. 106–112.

[5] Stelios D Bekiros. 2010. Heterogeneous trading strategies with adaptive fuzzy actor–critic reinforcement learning: A behavioral approach. *Journal of Economic Dynamics and Control* 34, 6 (2010), 1153–1170.

[6] Allan Borodin, Ran El-Yaniv, and Vincent Gogan. 2000. On the Competitive Theory and Practice of Portfolio Selection (Extended Abstract). In *Proceedings of LATIN (Lecture Notes in Computer Science)*, Vol. 1776. 173–196.

[7] Allan Borodin, Ran El-Yaniv, and Vincent Gogan. 2003. Can We Learn to Beat the Best Stock. In *Proceedings of NIPS*. 345–352.

[8] Zhengping Che, Sanjay Purushotham, Kyunghyun Cho, David Sontag, and Yan Liu. 2018. Recurrent neural networks for multivariate time series with missing values. *Scientific Reports* 8, 1 (2018), 1–12.

[9] Kyunghyun Cho, Bart van Merrienboer, Çaglar Gülçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. In *Proceedings of EMNLP*. 1724–1734.

[10] Marco Corazza and Francesco Bertoluzzo. 2014. Q-learning-based financial trading systems with applications. *University Ca'Foscari of Venice, Dept. of Economics Working Paper Series* 15 (2014), 1–23.

[11] Thomas M. Cover. 1991. Universal Portfolios. *Mathematical Finance* 1, 1 (1991), 1–29.

[12] Yue Deng, Feng Bao, Youyong Kong, Zhiquan Ren, and Qionghai Dai. 2017. Deep Direct Reinforcement Learning for Financial Signal Representation and Trading. *IEEE Transactions on Neural Networks and Learning Systems* 28, 3 (2017), 653–664.

[13] Thomas G Fischer. 2018. *Reinforcement learning in financial markets - a survey*. Technical Report. FAU Discussion Papers in Economics.

[14] Li Gao and Weiguo Zhang. 2013. Weighted moving average passive aggressive algorithm for online portfolio selection. In *Proceedings of IHMSC*, Vol. 1. 327–330.

[15] Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *Proc. of International Conference on Artificial Intelligence and Statistics (Proceedings of Machine Learning Research)*. 249–256.

[16] László Györfi, Gábor Lugosi, and Frederic Udina. 2006. Nonparametric kernel-based sequential investment strategies. *Mathematical Finance: An International Journal of Mathematics, Statistics and Financial Economics* 16, 2 (2006), 337–357.

[17] David P. Helmbold, Robert E. Schapire, Yoram Singer, and Manfred K. Warmuth. 1996. On-Line Portfolio Selection Using Multiplicative Updates. In *Proceedings of ICML*. 243–251.

[18] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Computation* 9, 8 (1997), 1735–1780.

[19] Dingjiang Huang, Junlong Zhou, Bin Li, Steven C. H. Hoi, and Shuigeng Zhou. 2016. Robust Median Reversion Strategy for Online Portfolio Selection. *IEEE Transactions on Knowledge and Data Engineering* 28, 9 (2016), 2480–2493.

[20] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. 2017. Densely Connected Convolutional Networks. In *Proceedings of CVPR*. 2261–2269.

[21] Zhengyao Jiang and Jinjun Liang. 2017. Cryptocurrency portfolio management with deep reinforcement learning. In *Proceedings of IntelliSys*. 905–913.

[22] Zhengyao Jiang, Dixing Xu, and Jinjun Liang. 2017. A deep reinforcement learning framework for the financial portfolio management problem. *arXiv preprint arXiv:1706.10059* (2017).

[23] Bin Li and Steven CH Hoi. 2012. On-line portfolio selection with moving average reversion. In *Proceedings of ICML*. 563–570.

[24] Bin Li, Steven C.H. Hoi, and Vivekanand Gopalkrishnan. 2011. CORN: Correlation-driven Nonparametric Learning Approach for Portfolio Selection. *ACM Transactions on Intelligent Systems and Technology* 2, 3 (2011), 21:1–21:29.

[25] Bin Li and Steven C. H. Hoi. 2014. Online portfolio selection: A survey. *Comput. Surveys* 46, 3 (2014), 35:1–35:36.

[26] Bin Li, Peilin Zhao, Steven C. H. Hoi, and Vivekanand Gopalkrishnan. 2012. PAMR: Passive aggressive mean reversion strategy for portfolio selection. *Machine Learning* 87, 2 (2012), 221–258.

[27] Malik Magdon-Ismail, Amir F Atiya, Amrit Pratap, and Yaser S Abu-Mostafa. 2004. On the maximum drawdown of a Brownian motion. *Journal of Applied Probability* 41, 1 (2004), 147–161.

[28] Harry Markowitz. 1952. Portfolio selection. *The Journal of Finance* 7, 1 (1952), 77–91.

[29] John Moody, Lizhong Wu, Yuansong Liao, and Matthew Saffell. 1998. Performance functions and reinforcement learning for trading systems and portfolios. *Journal of Forecasting* 17, 5-6 (1998), 441–470.

[30] Gordon Ritter. 2018. Reinforcement Learning in Finance. *Big Data and Machine Learning in Quantitative Investment* (2018), 225–250.

[31] David E Rumelhart, Geoffrey E Hinton, Ronald J Williams, and others. 1988. Learning representations by back-propagating errors. *Cognitive Modeling* 5, 3 (1988), 533–536.

[32] William F Sharpe. 1994. The sharpe ratio. *Journal of portfolio management* 21, 1 (1994), 49–58.

[33] David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin A. Riedmiller. 2014. Deterministic Policy Gradient Algorithms. In *Proceedings of ICML*. 387–395.

[34] Karen Simonyan and Andrew Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. In *Proceedings of ICLR*. 1–14.

[35] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott E. Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. 2015. Going deeper with convolutions. In *Proceedings of CVPR*. 1–9.

[36] Dat Thanh Tran, Alexandros Iosifidis, Juho Kanniainen, and Moncef Gabbouj. 2019. Temporal Attention-Augmented Bilinear Network for Financial Time-Series Data Analysis. *IEEE Trans. on Neural Networks and Learning Systems* 30, 5 (2019), 1407–1418.